

Faculty of Information Technology (FIT) – Ho Chi Minh City University of Science (HCMUS)

CS423 / CSC13003 – Software Testing (AI-augmented · 2026)

1. Student information

- **Student name (printed):** Trần Trí Nhân
- **Student ID:** 23127097
- **Class / Cohort:** Kiểm thử phần mềm - 23KTPM1
- **Assignment ID:** HW04
- **Assignment date:** 02/08/2026
- **AI tool(s) used:** Claude
- **AI tool(s) used:**
 - ☒ Yes
 - ☐ No

2. Audit Table

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
Tool: Claude Time: 00:55 02/08/2026 Input files: Register.jsx, test-cases-fr01.csv Prompt: "I will give you a list of test cases. Write Playwright automation script for these test cases on the Register page. The URL for the page is: http://localhost:5173/register Constraints: * All tests in a single file. * Use latest version("@playwright/test": "^1.62.0") of Playwright * Make the script data-driven. Use 'csv-parse' library("csv-parse": "^7.0.1") to read test data. Keep the test data csv file name blank, I will replace later. The csv columns of test data are: Data ID,Full name,Email,Password. Data ID format is: DTxx, for example: DT01 * Use at least 3 distinct Playwright assertion patterns * Include 'Run by: 23127097' in test.describe()	import { test, expect } from '@playwright/test'; import { parse } from 'csv-parse/sync'; import fs from 'fs'; import path from 'path'; const BASE_URL = 'http://localhost:5173/register'; const CSV_FILE = ""; interface TestRow { 'Data ID': string; 'Full name': string; Email: string; Password: string; } const csvContent = fs.readFileSync(path.resolve(__dirname, CSV_FILE), 'utf-8'); const rows: TestRow[] = parse(csvContent, { columns: true, skip_empty_lines: true, }); const WEAK_PASSWORD_MSG = 'Mật khẩu quá yếu! Phải dài tối thiểu 8 ký tự, gồm chữ hoa, chữ thường, số và KÝ TỰ ĐẶC BIỆT.'; const REGISTER_FAIL_MSG = 'Đăng ký thất bại.'; // Maps Data ID (DT01..DT11) to the corresponding test case's expected outcome type Outcome = 'success' 'nativeOrFail' 'fail' 'nativeOrWeak' 'weak';	VALID	The generated tests used correct locators and ran correctly without any unexpected crashes, while included more than 3 different assertion patterns, satisfies the requirement for task 1.	Replaced the placeholder for test data csv file with correct path.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
* Keep the data-driven test cases(for-loop through csv file test cases) and normal test cases separate. First 11 test cases are data-driven test cases * Write in TypeScript * Keep the comments minimal * Generate the listed test cases only. Do not invent test case or modify the behavior, or the expected result, regardless of the frontend code. * Beware of unlinked labels, unbound test ids and unlinked accessible names, fallback to css locator in these cases Give the me the test file without further explanation. Do not generate test data file, I already have it."	<pre>const outcomeById: Record<string, Outcome> = { DT01: 'success', DT02: 'nativeOrFail', DT03: 'nativeOrFail', DT04: 'fail', DT05: 'fail', DT06: 'nativeOrWeak', DT07: 'weak', DT08: 'weak', DT09: 'weak', DT10: 'weak', DT11: 'weak', }; test.describe('Register Page - Data Driven Tests. Run by: 23127097', () => { for (const row of rows) { const dataId = row['Data ID']; const fullName = row['Full name']; const email = row['Email']; const password = row['Password']; const outcome = outcomeById[dataId]; test(`\${dataId} - Register with given data, async ({ page }) => { await page.goto(BASE_URL); const nameInput = page.locator('input[type="text"]').first(); const emailInput = page.locator('input[type="text"]').nth(1); const passwordInput = page.locator('input[type="password"]'); const submitButton = page.locator('button[type="submit"]'); if (fullName) await nameInput.fill(fullName); if (email) await emailInput.fill(email); if (password) await passwordInput.fill(password); await submitButton.click(); if (outcome === 'success') { await expect(page).toHaveURL(/login\$/); } else if (outcome === 'nativeOrFail') { const isInvalid = await nameInput.evaluate((el: HTMLInputElement) => !el.checkValidity()) => !el.checkValidity() .catch(() => false); const emailInvalid = await emailInput.evaluate((el: HTMLInputElement) => !el.checkValidity()) .catch(() => false); if (!fullName !email) { expect(isInvalid emailInvalid).toBeTruthy(); } else { await expect(page.locator('div.bg-red- 100')).toContainText(REGISTER_FAIL_MSG); } } else if (outcome === 'fail') { await expect(page.locator('div.bg-red-</pre>			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
	<pre> 100')).toContainText(REGISTER_FAIL_MSG); } else if (outcome === 'nativeOrWeak') { const passwordInvalid = await passwordInput.evaluate((el: HTMLInputElement) => !el.checkValidity()).catch(() => false); if (!password) { expect(passwordInvalid).toBeTruthy(); } else { await expect(page.locator('div.bg-red-100')).toHaveText(WEEK_PASSWORD_MSG); } } else if (outcome === 'weak') { await expect(page.locator('div.bg-red-100')).toHaveText(WEEK_PASSWORD_MSG); } }); } }); </pre> <pre> test.describe('Register Page - Normal Tests. Run by: 23127097', () => { test('TC-12 - Verify confirm password field exists', async ({ page }) => { await page.goto(BASE_URL); const confirmPasswordLabel = page.locator('label', { hasText: 'Xác nhận mật khẩu' }); await expect(confirmPasswordLabel).toBeVisible(); }); }); </pre>			
Tool: Claude Time: 14:43 02/08/2026 Input files: App.jsx(from frontend-web), Cart.jsx, Home.jsx, ProductDetail.jsx, test-cases-fr07.csv Prompt: "I will give you a list of test cases. Write Playwright automation script for these test cases. The base URL for the page is: http://localhost:5173 Constraints: * All tests in a single file. * Use latest version("@playwright/test": "^1.62.0") of Playwright * Make the script data-driven. Use 'csv-parse' library("csv-parse": "^7.0.1") to read test data. Keep the test data csv file name blank, I will replace later. The csv columns of test data are:	<pre> import { test, expect } from '@playwright/test'; import { parse } from 'csv-parse/sync'; import * as fs from 'fs'; import * as path from 'path'; const BASE_URL = 'http://localhost:5173'; interface CsvRow { 'Data ID': string; 'Quantity': string; 'Item Name': string; } test.describe('FR-07 Shopping Cart Tests - Run by: 23127097', () => { test('TC-01 Verify shopping cart page has correct columns', async ({ page }) => { await page.goto(BASE_URL); await page.locator('button:has-text("Thêm vào giỏ")').first().click(); await page.locator('a:has-text("Giỏ hàng")').click(); const headerRow = page.locator('table thead tr'); await expect(headerRow).toContainText('Sản phẩm'); await expect(headerRow).toContainText('Đơn giá'); </pre>	VALID	The generated tests used correct locators and ran correctly without any unexpected crashes, while included more than 3 different assertion patterns, satisfies the requirement for task 1.	Replaced the placeholder for test data csv file with correct path.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
Data ID,Quantity,Item Name. Data ID format is: DTxx, for example: DT01 * Use at least 3 distinct Playwright assertion patterns * Include 'Run by: 23127097' in test.describe() * Keep the data-driven test cases(for-loop through csv file test cases) and normal test cases separate. The last 7 test cases are data-driven test cases * Write in TypeScript * Keep the comments minimal * Generate the listed test cases only. Do not invent test case or modify the behavior, or the expected result, regardless of the frontend code. * Beware of unlinked labels, unbound test ids and unlinked accessible names, fallback to css locator in these cases Give the me the test file without further explanation. Do not generate test data file, I already have it."	<pre> await expect(headerRow).toContainText('Số lượng'); await expect(headerRow).toContainText('Thành tiền'); await expect(headerRow).toContainText('Thao tác'); await expect(headerRow.getByText('+', { exact: true })).toBeVisible(); await expect(headerRow.getByText('-', { exact: true })).toBeVisible(); }); test('TC-02 Verify shopping cart not add new row when adding the same product', async ({ page }) => { await page.goto(BASE_URL); await page.locator('button:has-text("Thêm vào giỏ")').first().click(); await page.locator('button:has-text("Thêm vào giỏ")').first().click(); await page.locator('a:has-text("Giỏ hàng")').click(); const rows = page.locator('table tbody tr'); await expect(rows).toHaveCount(1); }); test('TC-03 Verify a back-to-home button exists on shopping cart page', async ({ page }) => { await page.goto(BASE_URL); await page.locator('a:has-text("Giỏ hàng")').click(); const backLink = page.locator('a:has-text("Tiếp tục mua sắm")'); await expect(backLink).toBeVisible(); }); test('TC-04 Verify total amount has correct label', async ({ page }) => { await page.goto(BASE_URL); await page.locator('button:has-text("Thêm vào giỏ")').first().click(); await page.locator('a:has-text("Giỏ hàng")').click(); await expect(page.locator('body')).not().toContainText('Tổng tạm tính'); const totalLabel = page.locator('text=Tổng cộng'); await expect(totalLabel).toBeVisible(); }); test('TC-05 Verify empty cart has clear message', async ({ page }) => { await page.goto(BASE_URL); await page.locator('a:has-text("Giỏ hàng")').click(); await expect(page.getByText('Giỏ hàng của bạn đang trống')).toBeVisible(); }); // Data-driven test cases const csvPath = path.join(__dirname, ''); const csvContent = fs.readFileSync(csvPath, 'utf-8'); const records: CsvRow[] = parse(csvContent, { columns: true, </pre>			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
	<pre> skip_empty_lines: true, }); test.describe('FR-07 Shopping Cart Quantity Validation (Data-Driven) - Run by: 23127097', () => { for (const record of records) { const dataId = record['Data ID']; const quantity = record['Quantity']; test(`\${dataId} Verify quantity input value: \${quantity}`, async ({ page }) => { await page.goto(BASE_URL); await page.locator('a:has-text("Xem chi tiết")').first().click(); const quantityInput = page.locator('input[type="number"]'); await quantityInput.fill(quantity); const addButton = page.locator('button:has-text("Thêm vào giỏ hàng")'); await addButton.click(); await addButton.click(); await page.locator('a:has-text("Giỏ hàng")').click(); const numericQuantity = Number(quantity); const isValid = Number.isInteger(numericQuantity) && numericQuantity > 0; if (isValid) { const rows = page.locator('table tbody tr'); await expect(rows).toHaveCount(1); const quantityCell = rows.first().locator('td').nth(2); await expect(quantityCell).toHaveText(String(numericQuantity)); } else { await expect(page.getByText('Giỏ hàng của bạn đang trống')).toBeVisible(); } }); } }); </pre>			
Tool: Claude Time: 15:47 02/08/2026 Input files: App.jsx(from frontend-admin), test-cases-fr14.csv Prompt: "I will give you a list of test cases. Write Playwright automation script for these test cases. The base URL for the page is: http://localhost:5174 The Admin account is: Email: admin@eshop.com Password: Admin123!	<pre> import { test, expect } from "@playwright/test"; import { parse } from "csv-parse/sync"; import fs from "fs"; import path from "path"; const BASE_URL = "http://localhost:5174"; const ADMIN_EMAIL = "admin@eshop.com"; const ADMIN_PASSWORD = "Admin123!"; interface CategoryTestData { "Data ID": string; "Category Name": string; } </pre>	VALID	The generated tests used correct locators and ran correctly without any unexpected crashes, while included more than 3 different	Replaced the placeholder for test data csv file with correct path.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
Constraints: * All tests in a single file. * Use latest version("@playwright/test": "^1.62.0") of Playwright * Make the script data-driven. Use 'csv-parse' library("csv-parse": "^7.0.1") to read test data. Keep the test data csv file name blank, I will replace later. The csv columns of test data are: Data ID,Category Name. Data ID format is: DTxx, for example: DT01 * Use at least 3 distinct Playwright assertion patterns * Include 'Run by: 23127097' in test.describe() * Keep the data-driven test cases(for-loop through csv file test cases) and normal test cases separate. The last 9 test cases are data-driven test cases * Write in TypeScript * Keep the comments minimal * Generate the listed test cases only. Do not invent test case or modify the behavior, or the expected result, regardless of the frontend code. * Beware of unlinked labels, unbound test ids and unlinked accessible names, fallback to css locator in these cases Give me the test file without further explanation. Do not generate test data file, I already have it."	<pre>const csvPath = path.join(__dirname, ""); const csvContent = fs.readFileSync(csvPath, "utf-8"); const testData: CategoryTestData[] = parse(csvContent, { columns: true, skip_empty_lines: true, }); async function loginAsAdmin(page: import("@playwright/test").Page) { await page.goto(BASE_URL); await page.getByPlaceholder("Email").fill(ADMIN_EMAIL); await page.getByPlaceholder("Password").fill(ADMIN_PASSWORD); await page.getByRole("button", { name: "Login" }).click(); } async function goToCategories(page: import("@playwright/test").Page) { await page.locator("li", { hasText: "Danh mục" }).click(); } test.describe("Category Management - Run by: 23127097", () => { test.beforeEach(async ({ page }) => { await loginAsAdmin(page); await goToCategories(page); }); test("TC-01 Verify admin can view categories", async ({ page }) => { const rows = page.locator("table tbody tr"); await expect(rows.first()).toBeVisible(); const rowCount = await rows.count(); expect(rowCount).toBeGreaterThan(0); await expect(rows.first().locator("button", { hasText: "Xóa" })).toBeVisible(); }); test("TC-02 Verify admin can delete a category", async ({ page }) => { const rows = page.locator("table tbody tr"); const initialCount = await rows.count(); const firstRowText = await rows.first().locator("td").nth(1).textContent(); page.once("dialog", (dialog) => dialog.accept()); await rows.first().locator("button", { hasText: "Xóa" }).click(); await expect(rows).toHaveCount(initialCount - 1); if (firstRowText !== null) { await expect(rows.first().locator("td").nth(1)).not.toHaveText(firstRowText); } }); test("TC-03 Verify delete button has a confirmation dialog", async ({ page }) => { let dialogAppeared = false;</pre>		assertion patterns, satisfies the requirement for task 1.	

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
	<pre> page.once("dialog", async (dialog) => { dialogAppeared = true; await dialog.dismiss(); }); const rows = page.locator("table tbody tr"); await rows.first().locator("button", { hasText: "Xóa" }).click(); await page.waitForTimeout(500); expect(dialogAppeared).toBeTruthy(); }); }); test.describe("Category Management Data-Driven - Run by: 23127097", () => { test.beforeEach(async ({ page }) => { await loginAsAdmin(page); await goToCategories(page); }); for (const row of testData) { const dataId = row["Data ID"]; const categoryName = row["Category Name"]; test(`\${dataId} - Category creation validation with input "\${categoryName}", async ({ page }) => { const rows = page.locator("table tbody tr"); const initialCount = await rows.count(); await page.locator('input[placeholder="Tên danh mục mới"]').fill(categoryName); await page.locator("button", { hasText: "Thêm mới" }).click(); await page.waitForTimeout(500); switch (dataId) { case "DT01": // TC-04: valid category name case "DT02": // TC-05: Vietnamese name case "DT03": // TC-06: numbers and symbols case "DT04": // TC-07: special characters await expect(rows).toHaveCount(initialCount + 1); await expect(page.locator("table tbody")).toContainText(categoryName); break; case "DT05": // TC-08: empty name rejected case "DT06": // TC-09: whitespace name rejected case "DT07": // TC-10: duplicate name rejected await expect(rows).toHaveCount(initialCount); break; case "DT08": { // TC-11: long name truncated with ellipsis await expect(rows).toHaveCount(initialCount + 1); const newCell = rows.last().locator("td").nth(1); const overflowStyle = await newCell.evaluate((el) => getComputedStyle(el).textOverflow, </pre>			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
	<pre>); expect(overflowStyle.toBe("ellipsis")); break; } case "DT09": { // TC-12: sanitize category name, rendered as text not HTML await expect(rows).toHaveCount(initialCount + 1); const newCell = rows.last().locator("td").nth(1); await expect(newCell).toHaveText(categoryName); const childElementCount = await newCell.evaluate(el => el.children.length,); expect(childElementCount).toBe(0); break; } } }); } });</pre>			

3. Summary of AI Accuracy

Metric	Count	Percentage
Total AI-generated artifacts audited	3	
VALID (correct, accepted as-is)	3	100%
INVALID (wrong; rejected)	0	0%
INCOMPLETE (acceptable after edits)	0	0%

4. Conclusion

In conclusion, Claude did an amazing work of generating Playwright test scripts for automation test cases in HW04. While the generated tests used correct locators and ran correctly without any unexpected crashes, the code quality itself was not ideal for the long run when the number of automation test cases increase, but in the end it was still acceptable given the scope of the software under testing (EShop) and the requirements in HW04. In addition to that, in a generated script, the AI completely ignored the name of the item when creating the for loop to iterate through the data-driven test cases. While this did not make the test results invalid, it was certainly better if the given data was used fully. By doing so, the tests would be less fragile and provide higher confidence about the run results.

5. Mandatory Disclosure

The test scripts were initially generated by Claude; I reviewed and modified the test data csv file path; the automation script review, the bug report and the test summary was written entirely by me. The HTML reports correctly contain "Run by: {StudentID}" together with ISO timestamp. The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category.

Signature

- **Student name (printed):** Trần Trí Nhân
- **Student ID:** 23127097

- **Class / Cohort:** Kiểm thử phần mềm - 23KTPM1
- **Course:** CS423 / CSC13003 – Software Testing
- **Instructor:** Lâm Quang Vũ, Hồ Tuấn Thanh, Trương Phước Lộc
- **Date:** 02/08/2026
- **Signature:**



Reference

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (latest version).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.
- DeepEval & Promptfoo documentation — testing frameworks for LLM systems.